

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и
автоматизированных систем

«КУРСОВАЯ РАБОТА»

по дисциплине: «Основы программирования»

тема: «Splay tree»

Автор работы _____ Тутов Данил Андреевич

(Подпись)

Руководитель работы _____ Притчин Иван Сергеевич

(Подпись)

Оценка _____

Белгород 2022 г.

Оглавление

Введение	2
1 Теоретическое введение	3
1.1 История возникновения	3
1.2 Определение	3
1.3 Идея алгоритма	4
1.3.1 Случай zig	4
1.3.2 Случай zig-zig	5
1.3.3 Случай zig-zag	6
2 Программная реализация	7
2.1 Реализация splay	7
2.2 Реализация интерфейса	9
2.2.1 Операция вставки	9
2.2.2 Операция поиска	10
2.2.3 Операция удаления	10
2.2.4 Операция объединения	13
2.2.5 Операция разделения	15
3 Анализ операции Splay	17
3.1 Операция zig	18
3.2 Операция zig-zag	19
3.3 Операция zig-zig	20
3.4 Общее заключение	21
Заключение	22
Список использованной литературы	23

Введение

"Структуры данных". Что же это за волшебное словосочетание, без которого не могла бы существовать жизнь программиста? Структура данных - общее свойство информационного объекта, с которым взаимодействует та или иная программа.

Общее свойство характеризуется:

1. множеством допустимых значений
2. набором допустимых операций
3. характером организованности

Структуры данных используются для эффективного хранения, извлечения и обработки данных. С их помощью можно представлять различные типы информации, включая списки, графики и деревья. Как вы можете понять, это фундаментальная часть программирования. Существует множество структур данных, однако не все могут справляться с большим количеством требований одновременно. Всегда есть, куда стремиться, поэтому постоянно придумываются всё новые, более усовершенствованные, эффективные структуры данных. Понимание того, как их эффективно использовать, может помочь вам решить сложные проблемы и сделать ваши программы более быстрыми.

В данной работе будет рассмотрена такая структура данных, как *Splay дерево*.

Перед началом работы над курсовым проектом я обозначил цели, а именно: изучить принципы СД Splay дерево; реализовать операции для взаимодействия (интерфейс); проанализировать сложность операций.

Глава 1

Теоретическое введение

1.1 История возникновения

Splay дерево, которое также называется расширяющимся, косым, растянутым, выворачивающимся, деревом отображения, развёрнутым, было придумано в середине восьмидесятых Робертом Тарьяном и Даниэлем Слейтором. Тысяча и одно название появились благодаря некорректному переводу с английского языка. Думаю, после рассмотрения принципов этой структуры данных, подходящее название вы сможете выбрать сами. Роберт и Даниэль предложили несколько красивых и эффективных структур данных, все они имели несложную базовую структуру и одну-две эвристики, которые их постоянно подправляют. Splay дерево - одна из таких структур.

1.2 Определение

Splay tree – бинарное дерево поиска с дополнительным свойством, позволяющим быстро получить доступ к тем элементам, к которым недавно обращались. Как и самобалансирующиеся бинарные деревья поиска, Splay дерево выполняет основные операции, такие как вставка, поиск и удаление за $O(\log N)$ амортизированного времени.

Для начала изучения нашей структуры данных, необходимо познать фундамент. Как было сказано ранее, Splay дерево – бинарное дерево поиска. Сделаем небольшое вступление в эту структуру, чтобы лучше понимать концепцию происходящего.

Бинарное дерево поиска (англ. *Binary search tree*) – двоичное дерево, для которого выполняется следующее свойство: у всех узлов левого поддерева произвольного узла X значения ключей данных *меньше либо равны*, нежели значение ключа самого узла X ; у всех узлов правого поддерева произвольного узла X значения ключей данных *больше*, нежели значение ключа данных самого узла X . На рисунке 1.1 можно наблюдать иллюстрацию бинарного дерева поиска.

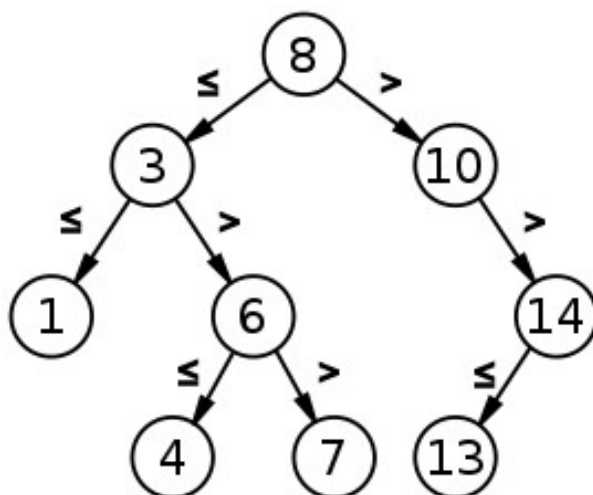


Рис. 1.1: Пример бинарного дерева поиска

1.3 Идея алгоритма

Перейдём к Splay дереву. Напоминаю, у этого дерева есть дополнительное свойство, позволяющее быстро получить доступ к тем элементам, к которым недавно обращались. Каким же образом можно реализовать такую возможность? Всё верно, необходимо наш "недавний" элемент продвигать в корень. Рассмотрим главную операцию – *splay*. Splay совершает повороты, при этом чередует различные виды, благодаря чему достигается логарифмическая амортизированная оценка. Данный алгоритм делится на три случая: *zig*, *zig-zig*, *zig-zag*. Каждая из этих ситуаций делится ещё на две, так как вершина, которую мы будем поднимать вверх по дереву, может быть как левым потомком, так и правым. Большого различия нет, алгоритмы будут симметричны, необходимо лишь понять основную концепцию таких поворотов.

1.3.1 Случай *zig*

Рассмотрим первый случай, который называется *zig*. Здесь вершина x – целевая вершина, именно её мы будем поднимать в корень дерева. Вершина p – корень дерева, а также родитель x . А всё то, что обозначено цифрами в маленьких кружочках – поддеревья. Несложно догадаться, что за их порядком тоже важно следить (основное свойство бинарного дерева поиска всё ещё присутствует). Совершается оборот ребра, соединяющего p и x , таким образом, теперь x является корнем, а p его правым потомком. Поддерево под номером 2 из правого ребёнка x превратилось в левого ребёнка p . Ситуация изображена на рис. 1.2

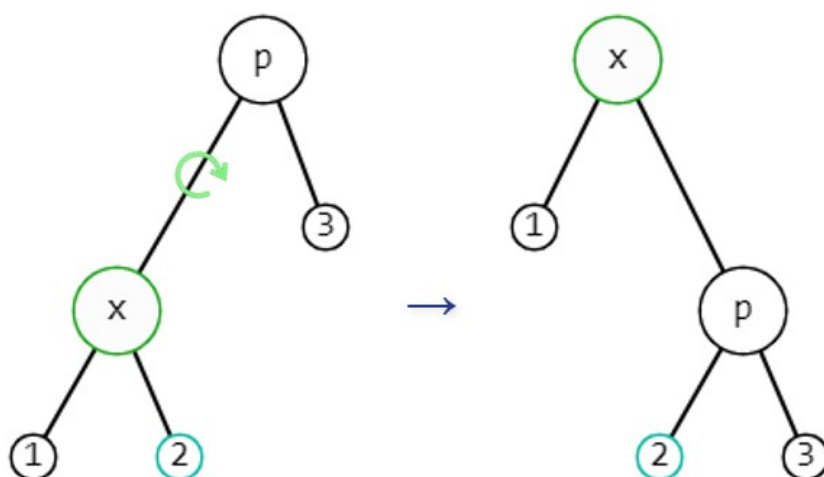


Рис. 1.2: Случай zig

Для лучшего осознания приведу пример с реальными вершинами (рис. 1.3). Заметьте, основная концепция бинарного дерева поиска, а именно: ключи левого поддерева $\leq$ локального корня; ключи правого поддерева $>$ локального корня, не пропала.

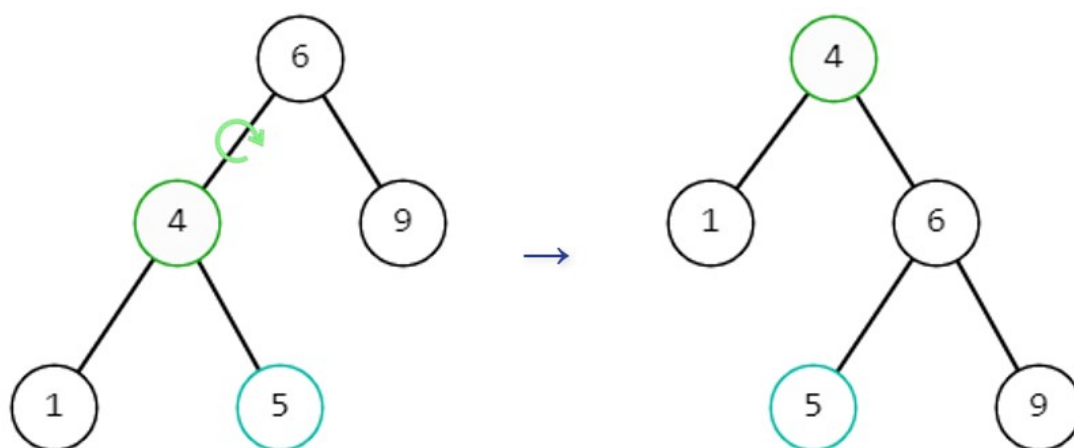


Рис. 1.3: Пример для случая zig

1.3.2 Случай zig-zig

Берём предыдущий случай, добавляем к родителю целевой вершины родителя – узел g . Таким образом, теперь g является корнем дерева. Данная операция (случай) выполняется тогда, когда p и x одновременно либо левые, либо правые дети. Есть одна важная особенность, сначала выполняется поворот по ребру (g, p) , затем по ребру (p, x) .

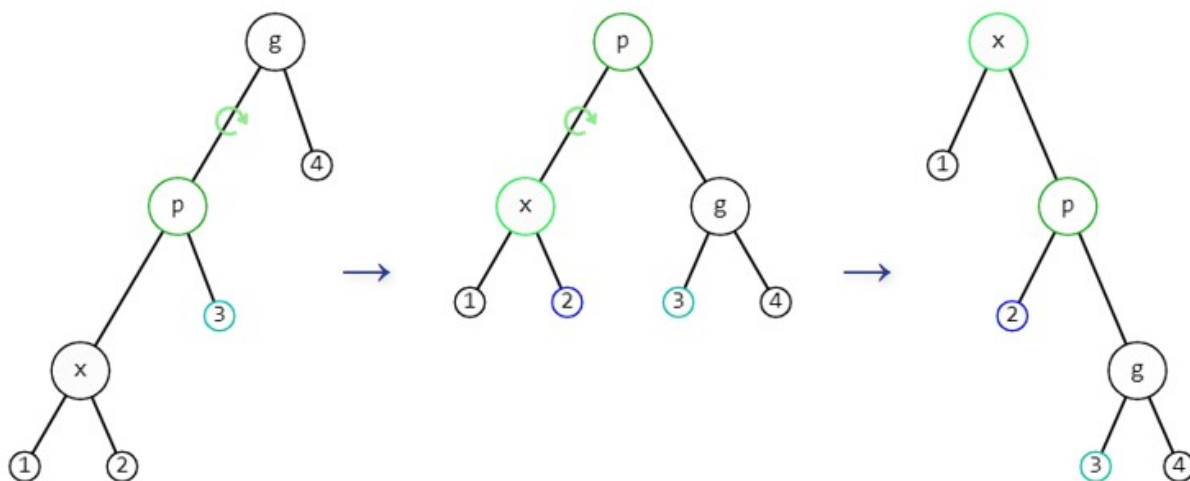


Рис. 1.4: Случай zig-zig

1.3.3 Случай zig-zag

Одна отличительная особенность, которую можно подметить из названия: здесь p и x являются детьми по разную сторону. Я имею в виду, если x левый ребёнок, то p правый, и наоборот. Выполняется поворот по ребру (p, x) , затем по ребру (g, x) .

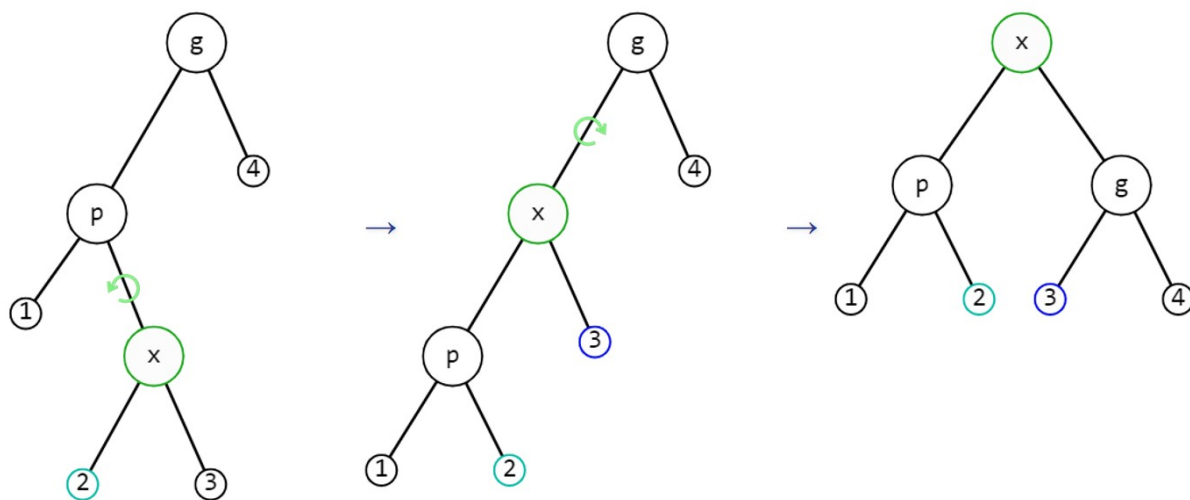


Рис. 1.5: Случай zig-zig

Вот мы и разобрали основу данной структуры данных. На этих принципах строится фундамент Splay tree. Перейдём же к программной реализации.

Глава 2

Программная реализация

2.1 Реализация splay

Начнём, конечно же, с самой реализации функции *splay*, которая и будет "выворачивать" нашу вершину в корень дерева. Объявим вспомогательные функции:

1. *leftRotate(n)* - выполняет поворот ребра, соединяющего вершину *n* и её правого ребёнка
2. *rightRotate(n)* - выполняет поворот ребра, соединяющего вершину *n* и её левого ребёнка

Опишем работу *rightRotate(n)* (графическое представление можно наблюдать на всё том же случае *zig*: рис. 1.2). На вход функция получает вершину и будет делать поворот ребра, соединяющего эту вершину с левым ребёнком, которого нужно запомнить (строка 3). Затем нам необходимо перебросить правого ребёнка левого ребёнка к родителю (строка 7) и сделать его левым ребёнком. На 12 строке располагается функция *updateParents*, которая обновляет указатели на сына для дедушки и на родителя для сына. Так как у нас отсутствует дедушка, теперь *x* является корнем дерева. Далее на 15 и 16 строчках мы обновляем указатели для прошлого ребёнка и прошлого родителя. Таким образом, теперь они у нас поменяны местами и знают свой порядок в этом семейном древе. Для наилучшего восприятия работы функции рекомендую самостоятельно "пройтись" по каждой строчке кода и в течение "прохода" отображать выполнение в каком-нибудь графическом редакторе (листочек и ручка тоже отлично с этим справляются).


```

1 // performs right rotation (rotation of the edge connecting n and it's left child)
2 void rightRotate(Node *n) {
3     Node *leftChild = n->leftChild;
4
5     // execute shift right child of the n's left child to n's left child
6     // (even if equal nullptr)
7     n->leftChild = leftChild->rightChild;
8     if (leftChild->rightChild) {
9         leftChild->rightChild->parent = n;
10    }
11
12    updateParents(n, leftChild);
13
14    // link old child and old parent
15    leftChild->rightChild = n;
16    leftChild->rightChild->parent = leftChild;
17 }

```

Реализация rightRotate

Теперь же приступим к описанию самого алгоритма *splay* и его программной реализации. Несложно догадаться, что существует 6 вариантов событий, которые будут решаться с помощью трёх основных операций (zig, zig-zig, zig-zag). Исходы могут быть такими: наша вершина может быть левым и правым потомком. Внутри каждого такого случая есть ещё по три события:

- у вершины может не быть прародителя
- родитель вершины может быть левым ребёнком
- родитель вершины может быть правым ребёнком

Рассмотрим первый случай (строки 6-9): целевая вершина является левым потомком, также у неё нет прародителя. Данная ситуация – *zig*-операция (или же просто правый поворот, который мы рассматривали выше). Не стоит забывать основную концепцию функций поворота, они вращают ребро, которое соединяет переданный узел и его левого потомка, если это правый поворот, или его правого потомка, если это левый поворот. В таком случае, нам необходимо передать в функцию узел родителя (строка 9).

Затем у нас наблюдается ситуация, когда родитель вершины является левым ребёнком (строки 10-14). Такая ситуация решается благодаря двум правым поворотам или же по-другому операции *zig-zig*. Напоминаю, сначала производится правый поворот ребра, соединяющего прародителя и родителя, а затем правый поворот родителя и целевой вершины.

И последняя ветка *else* (строки 15-19), которая означает, что предок нашей вершины правый потомок. Здесь необходима операция *zig-zag*. После двадцать первой строки идёт рассмотр симметричных случаев (целевая вершина - правый потомок).

Советую ещё раз взглянуть на картинки представленных случаев, а затем мысленно продумать, как будут работать последовательности из определённых поворотов.

```

1 // performs splay
2 // (delivers a node n to the root of the tree using left and right pivots)
3 void splay(Node *n) {
4     while (parent(n)) {
5         // case: node n is left child
6         if (n == parent(n)->leftChild) {
7             if (!grandparent(n)) { // there isn't grandparent
8                 // zig operation
9                 rightRotate(parent(n));
10            } else if (grandparent(n)->leftChild == parent(n)) {
11                // n's parent - left child
12                // zig-zig operation
13                rightRotate(grandparent(n));
14                rightRotate(parent(n));
15            } else {
16                // n's parent - right child
17                // zig-zag operation
18                rightRotate(parent(n));
19                leftRotate(parent(n));
20            }
21        } else { // case: node n is right child
22            if (!grandparent(n)) { // there isn't grandparent
23                // zig operation
24                leftRotate(parent(n));
25            } else if (grandparent(n)->rightChild == parent(n)) {
26                // n's parent - right child
27                // zig-zig operation
28                leftRotate(grandparent(n));
29                leftRotate(parent(n));
30            } else { // n's parent - left child
31                // zig-zag operation
32                leftRotate(parent(n));
33                rightRotate(parent(n));
34            }
35        }
36    }
37 }

```

Реализация Splay

2.2 Реализация интерфейса

Изучим основные операции Splay дерева. В данной структуре есть: вставка, поиск, удаление, объединение и разделение. Как уже говорилось ранее, фундаментом этой СД является бинарное дерево поиска, поэтому, как ни странно, основные функции будут сильно походить на функции из дерева поиска (но могут быть и другие эвристики).

2.2.1 Операция вставки

Операцию вставки можно реализовать двумя способами.

1. Вариант через бинарное дерево поиска

- Используем алгоритм для вставки из дерева поиска
- Выполняем операцию *Splay* для вставленного узла, в результате чего добавленный узел становится корнем дерева

2. Вариант через функцию разделения

- Разбиваем дерево по значению x на два поддерева (T_1 , T_2)
- Создаём новое дерево, в котором x – корень, T_1 – левое поддерево, T_2 – правое поддерево

В своей реализации я организовал операцию вставки через алгоритмы для двоичного дерева поиска. Ознакомиться с кодом можно в приложении.

2.2.2 Операция поиска

Ничего сверхъестественного. Выполняется так же, как и в бинарном дереве поиска, только в конце запускается наша классическая операция *splay*.

2.2.3 Операция удаления

Удаление. Эта операция мне так понравилась, что я хочу разобрать каждую строчку её реализации, сопровождая объяснение графическими иллюстрациями. Создадим случайное дерево (рис. 2.1).

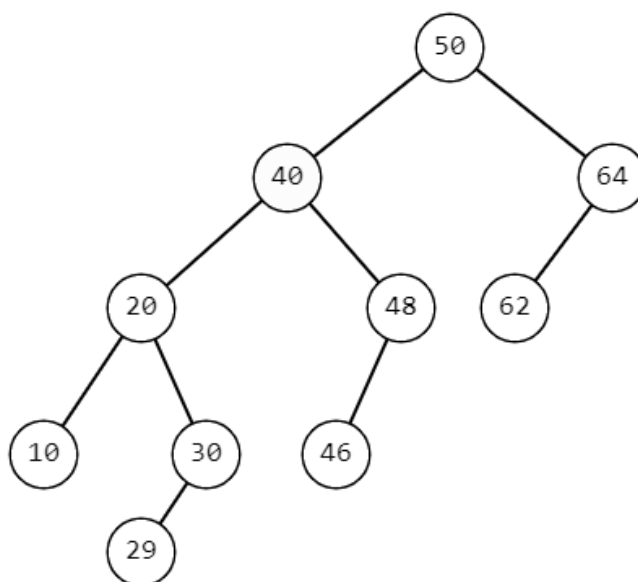


Рис. 2.1: Дерево

Выберем узел, который мы хотим удалить. Допустим этим узлом будет вершина с числом 40. Итак, начинаем рассуждать. Данная вершина имеет сразу два потомка, их необходимо как-то сохранить, а удалённой вершине найти замену. Узлом, заменяющим удаляемую вершину, может выступать:

1. предшественник¹ удаляемой вершины
2. преемник² удаляемой вершины

В моей реализации новым узлом вместо удалённого выступает предшественник.

```
1 Node* replacementRemotedNode = _maximum(deletingNode->leftChild);
```

¹самый правый узел левого поддерева

²самый левый узел правого поддерева

Хорошо, мы нашли замену (узел со значением 30), но ведь надо не забыть про один очень важный момент. Заменяемый узел может иметь потомка, причём только одного (по определению минимума/максимума). В моей реализации у *replacementRemotedNode* может быть левый потомок. Необходимо не забыть его сохранить.

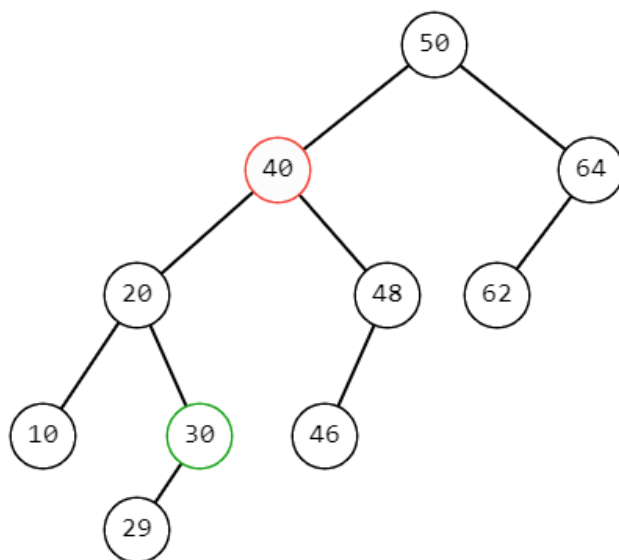


Рис. 2.2: Операция удаления. Шаг 2

В данной части кода мы: обмениваем максимум (замена удаляемому узлу) и его левого потомка между собой (строка 2); делаем левым ребёнком замены левого ребёнка удаляемого узла (строки 4-5).

```

1  if (parent(replacementRemotedNode) != deletingNode) {
2      updateParents(replacementRemotedNode, replacementRemotedNode->
        leftChild);
3
4      replacementRemotedNode->leftChild = deletingNode->leftChild;
5      replacementRemotedNode->leftChild->parent =
        replacementRemotedNode;
6  }

```

На данный момент мы имеем такую странную ситуацию.

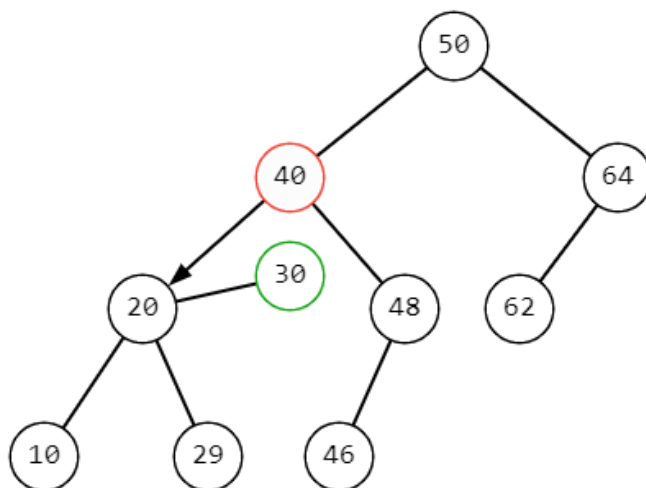


Рис. 2.3: Операция удаления. Шаг 3

Нам осталось поставить значение 30 вместо значения 40 . Для этого нужно:

1. переназначить у родителя удаляемого узла указатель на сына на заменяемый узел (строка 1)
2. прикрепить к заместителю правое поддерево удаляемого узла (строки 3-4)

```
1 updateParents(deletingNode, replacementRemotedNode);
2
3 replacementRemotedNode->rightChild = deletingNode->rightChild;
4 deletingNode->rightChild->parent = replacementRemotedNode;
```

Теперь об узле со значением 40 все забыли, как бы грустно это не было. Далее по алгоритму идёт операция *splay* для родителя удалённого узла, но так как его родитель и так был корнем дерева, то ничего нового не произойдёт.

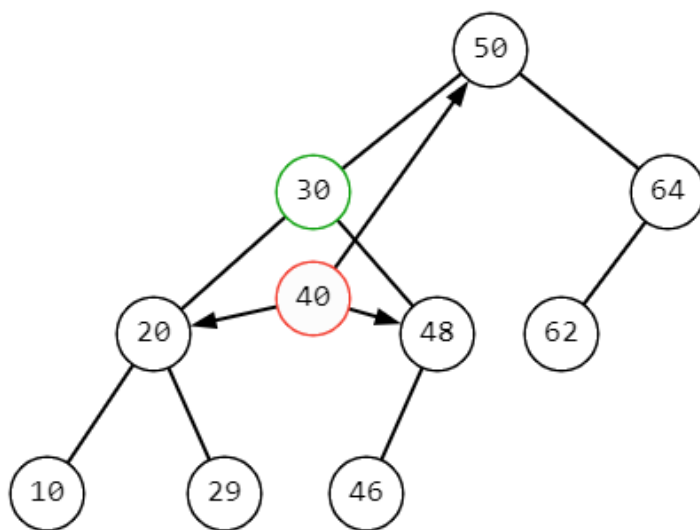


Рис. 2.4: Операция удаления. Шаг 4

Осталось освободить память, занимаемую удалённой вершиной, после чего мы будем иметь такую картину. На этом операция удаления завершена.

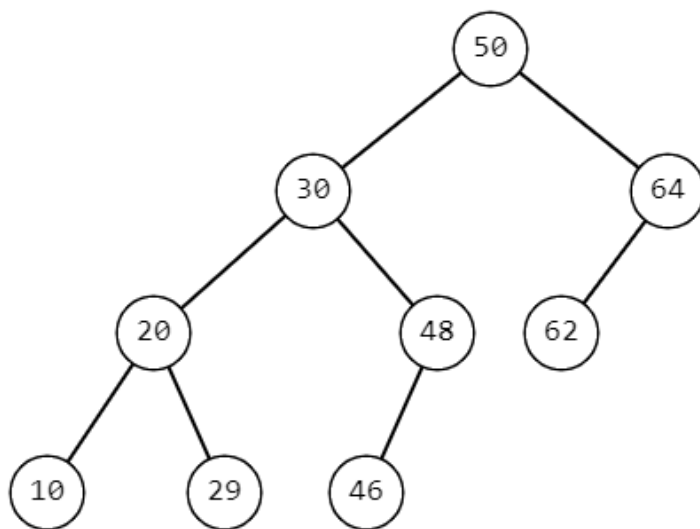


Рис. 2.5: Финальное дерево

2.2.4 Операция объединения

Для слияния двух деревьев (T_1 и T_2), в которых все ключи T_1 меньше ключей T_2 , выполняется функция *splay* для максимального элемента дерева T_1 . Таким образом, корень T_1 не имеет правого ребёнка. После этих действий, мы делаем дерево T_2 правым потомком T_1 .

Рассмотрим следующее дерево (рис. 2.6).

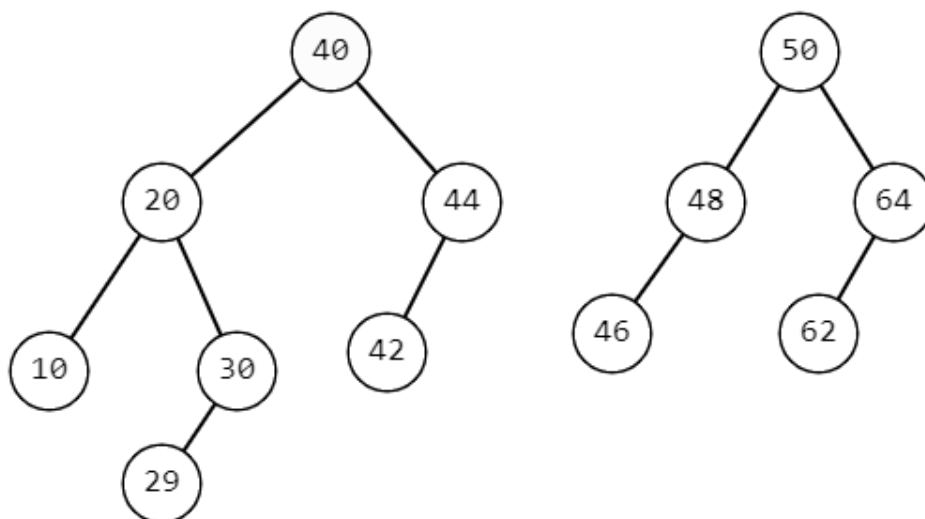


Рис. 2.6: Дерево

Находим максимальный элемент дерева с корнем 40. Это узел со значением 44. Выполняем операцию *splay* для этой вершины.

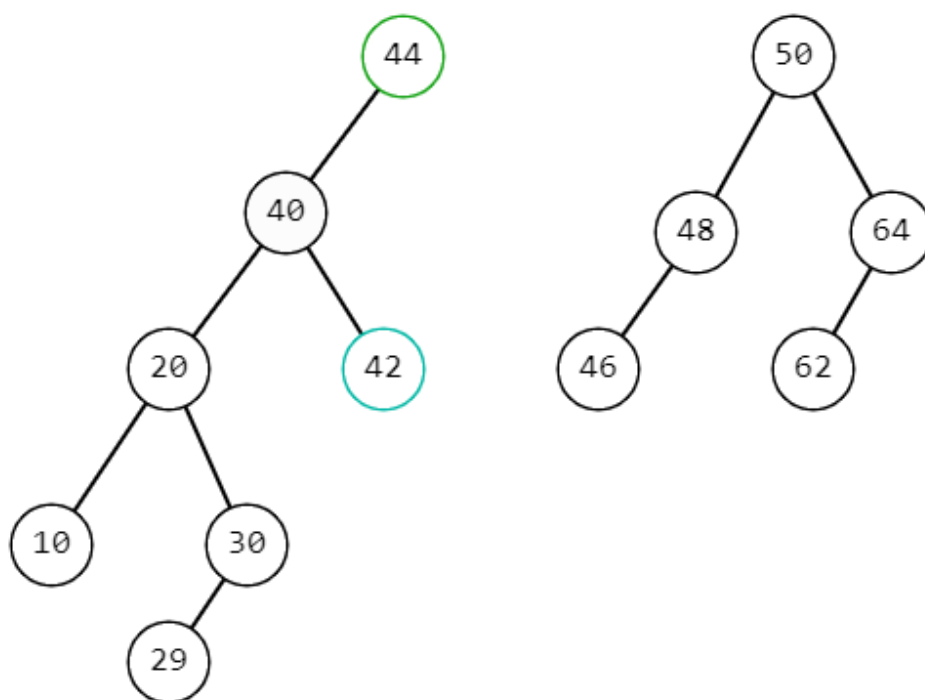


Рис. 2.7: Операция объединения. Шаг 2

После проделанных действий делаем дерево со значением в корне 50 правым потомком дерева с корнем 44 .

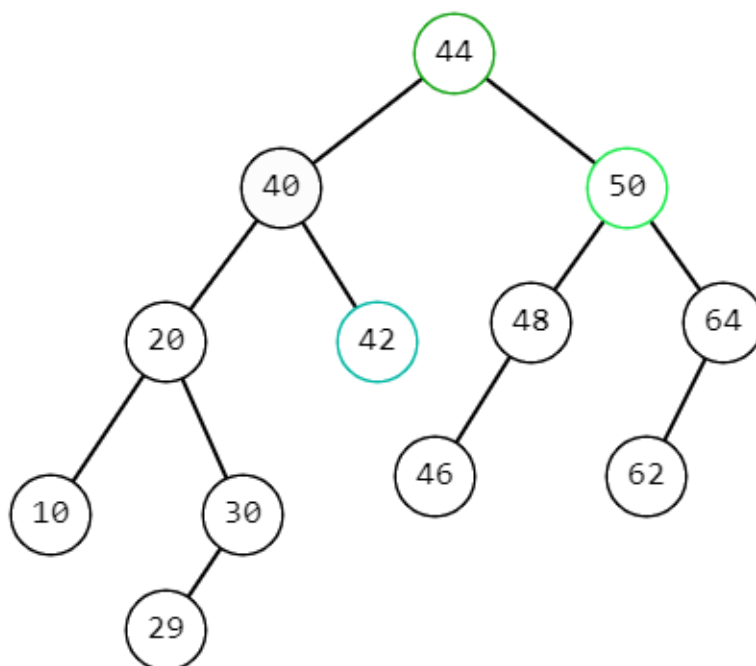


Рис. 2.8: Операция объединения. Шаг 3

В результате выполненных операции мы имеем одно дерево с корнем 44 .

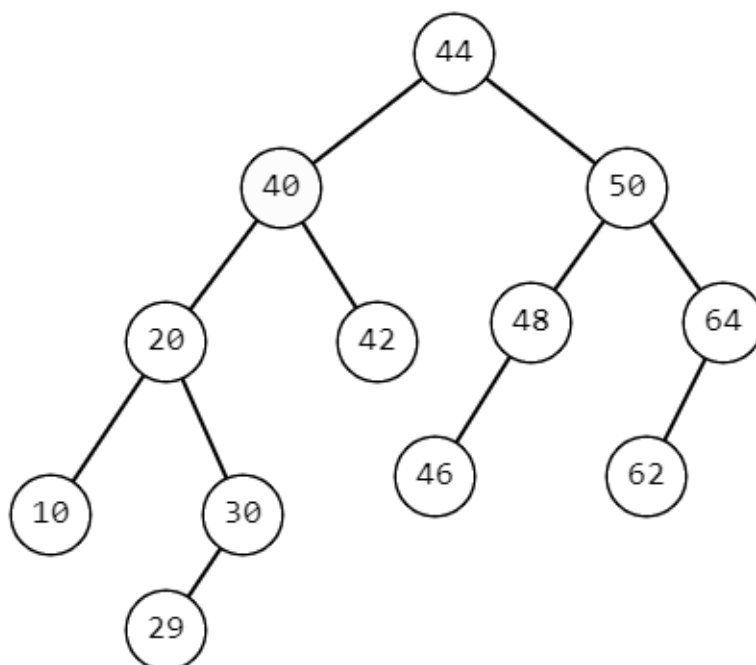


Рис. 2.9: Финальное дерево

2.2.5 Операция разделения

Разделение дерева реализуется так: находим ближайший элемент к искомому (или равный), затем делаем для него *splay*. Отделяем правого потомка от корня. Теперь мы имеем два дерева: первое содержит все элементы меньше искомого, второе элементы больше искомого.

Построили дерево (рис. 2.10).

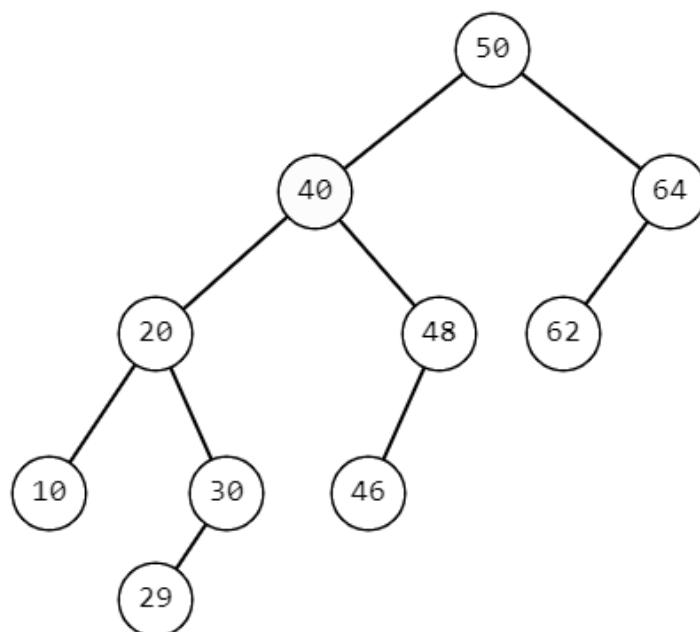


Рис. 2.10: Дерево

Нашли элемент, по которому будем разделять дерево (например, вершина с числом 40).

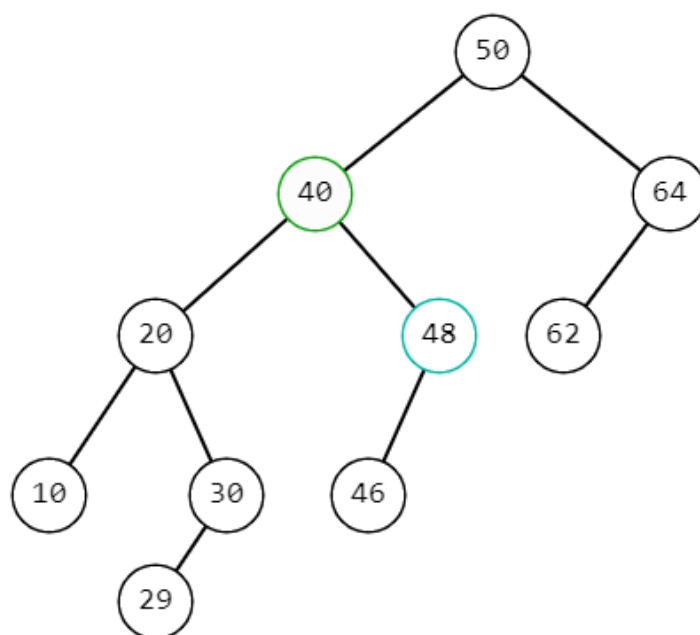


Рис. 2.11: Операция разделения. Шаг 2

Выполняем операцию *splay* для этого узла.

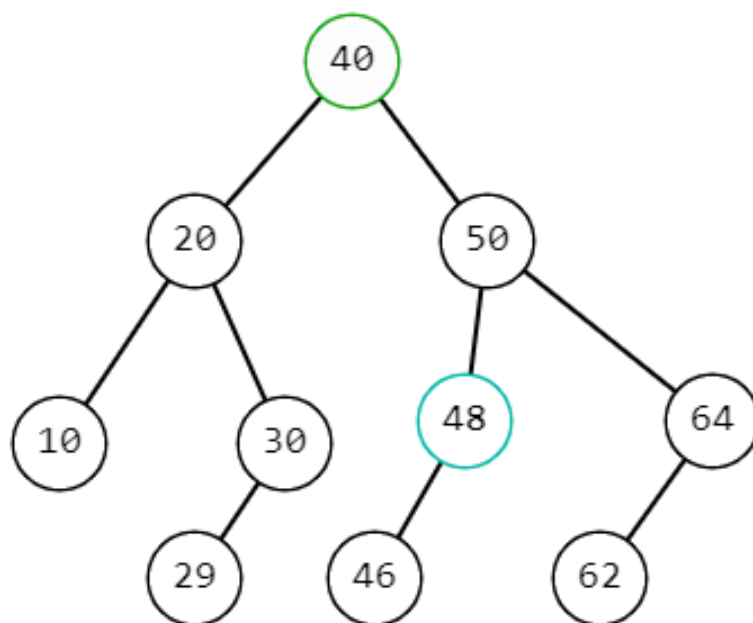


Рис. 2.12: Операция разделения. Шаг 3

Отделяем правого потомка от корня, получаем два дерева. Готово!

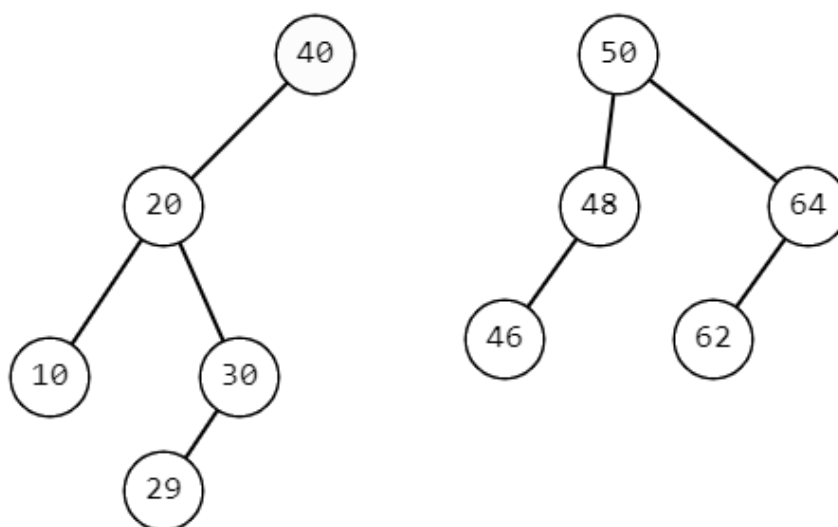


Рис. 2.13: Финальное дерево

Глава 3

Анализ операции Splay

Для чего мы делаем нашу легендарную операцию *splay* каждый раз, когда это возможно? Возьмём функцию *insert* и рассмотрим её абстрактно. Сначала мы спускаемся вниз по дереву, после того, как нашли место и вставили элемент, выполняем операцию *splay*. Сначала мы тратим какое-то количество времени, чтобы спуститься вниз, затем какое-то количество времени, чтобы выполнить *splay*.

Утверждение такое: время, которое мы потратили на спуск, равно времени выполнения операции *splay*. Таким образом, время любой операции: $T(\text{operation}) = O(T(\text{splay}))$. К слову, важно отметить, что время выполнения функции *splay* амортизированное¹. Докажем данное выражение: $\tilde{T}(\text{splay}) = O(\log N)$. Введём потенциал ϕ^2 . Возьмём все узлы нашего дерева, посчитаем для каждого узла размер поддеревы. $S(x)$ – размер поддеревы x . Ранг³ узла x : $r(x) = \log_2 S(x)$. Потенциал $\phi = \sum r(x)$.

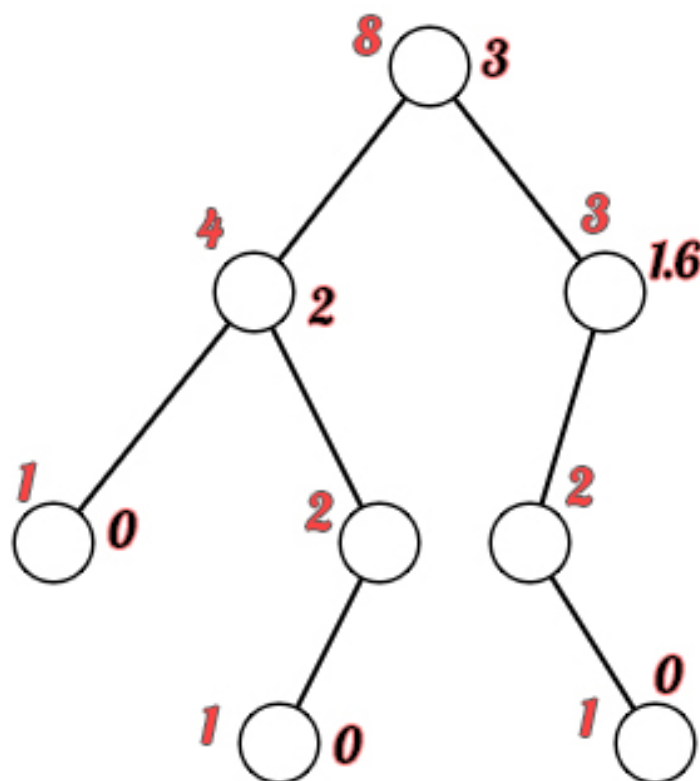
Итак, ниже представлено дерево. Определим размеры узлов (красным цветом) и ранги (чёрным цветом). Соответственно, потенциал ϕ этого дерева равен 8.6. Амортизированное время $\tilde{T} = T + \Delta\phi$, где T – настоящее время; $\Delta\phi$ – изменение потенциала. Докажем, что, если мы возьмём элемент x и сделаем операцию *splay*, $\tilde{T}(\text{splay}(x)) \leq 1 + 3(r'(x) - r(x))^4$. Новый ранг $x = \log_2 n$, так как он теперь в корне (после операции *splay*).

¹Основной идеей амортизационного анализа является то, что любая трудоёмкая операция меняет состояние программы таким образом, что до следующей трудоёмкой операции обязательно пройдёт достаточно много мелких, тем самым «амортизируя» вклад трудоёмкой операции.

²Простыми словами "степень плохости дерева". ϕ большой – плохо.

³Ранг дерева – количество узлов в дереве, которое определяет максимальное количество ребер между корнем и любым другим узлом дерева. В двоичном дереве поиска ранг дерева равен глубине дерева.

⁴ $r'(x)$ – новый ранг x ; $r(x)$ – старый ранг x .

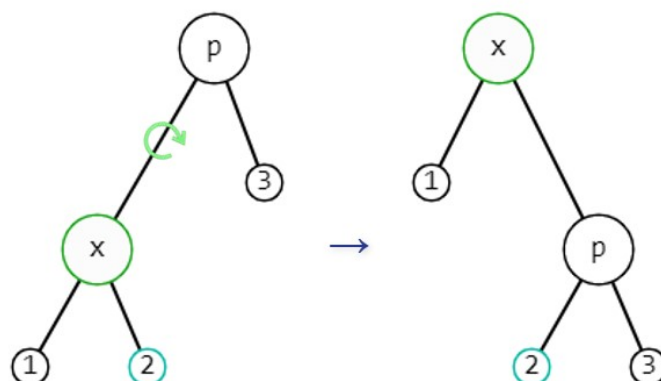


Операция *splay* состоит, как мы уже знаем, из нескольких операций. Докажем время выполнения каждой операции (zig; zig-zig; zig-zag).

1. Для $zig \leq 1 + 3(r'(x) - r(x))$
2. Для $zig-zig$ и $zig-zag \leq 3(r'(x) - r(x))^5$

3.1 Операция zig

Рассмотрим первую операцию.



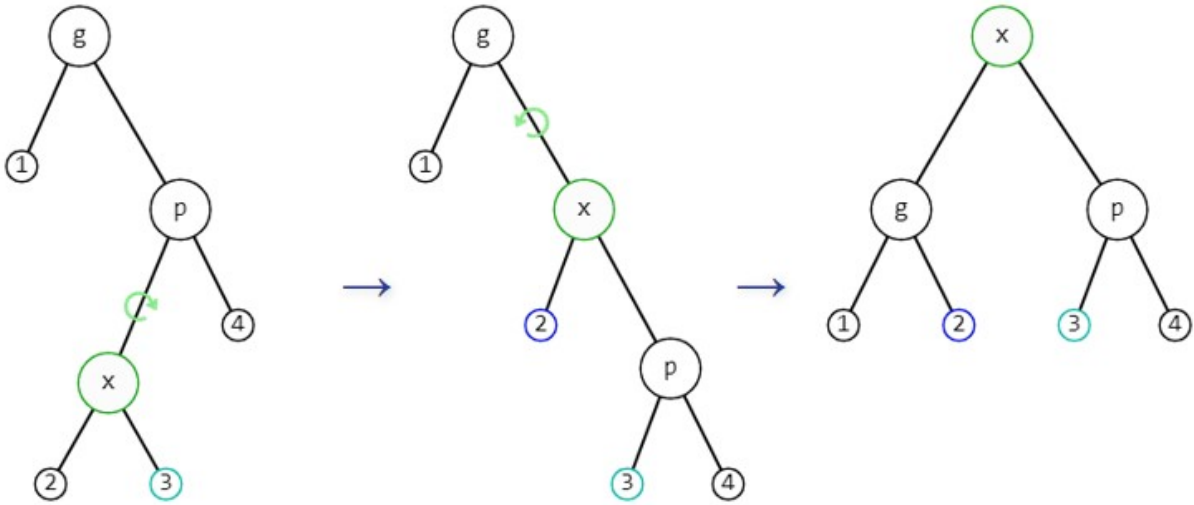
Мы помним, амортизированное время $\tilde{T} = T + \Delta\phi$. Таким образом $\tilde{T} = 1 + (r'(x) - r(x) + r'(p) - r(p))$. Теперь пояснения: единичка – настоящее время, да, мы будем счи-

⁵Если на данном этапе в голове появляются мысли на подобие 'что тут вообще происходит?', это нормально. Дальше должно быть лучше.

тать время в поворотах (один поворот ребра – одна операция), затем второе слагаемое – изменение потенциала. Ещё раз, потенциал – сумма всех рангов. Чтобы узнать, как изменилась сумма всех рангов, нужно узнать, у каких вершин поменялись ранги. Ответ очевиден: у вершин x и p . Отсюда такая формула для амортизированного времени.

Ранг p уменьшился, тогда $r'(p) - r(p) \leq 0$. Ранг x увеличился, тогда $r'(x) - r(x) \geq 0$. Таким образом, остаётся $1 + (r'(x) - r(x))$, а оно точно больше $1 + 3(r'(x) - r(x))$. Утверждение доказано.

3.2 Операция zig-zag



$$\tilde{T} = 2 + (r'(x) + r'(p) + r'(g) - r(x) - r(p) - r(g)).$$

2 – настоящее время (измеряемое в количествах поворотов),

выражение в скобках – изменение потенциала (только у этих трёх вершин поменялся ранг).

Итак, преобразуем выражение:

- у старой вершины g и новой вершины x размеры одинаковые, можно убрать $r'(x)$ и $r(g)$
- $r(p) \geq r(x)$, можно заменить $-r(p)$ на $-r(x)$ ⁶

Получается следующее: $\tilde{T} = 2 + (r'(p) + r'(g) - 2r(x))$. Докажем, что это выражение $\leq 2(r'(x) - r(x))$ ⁷.

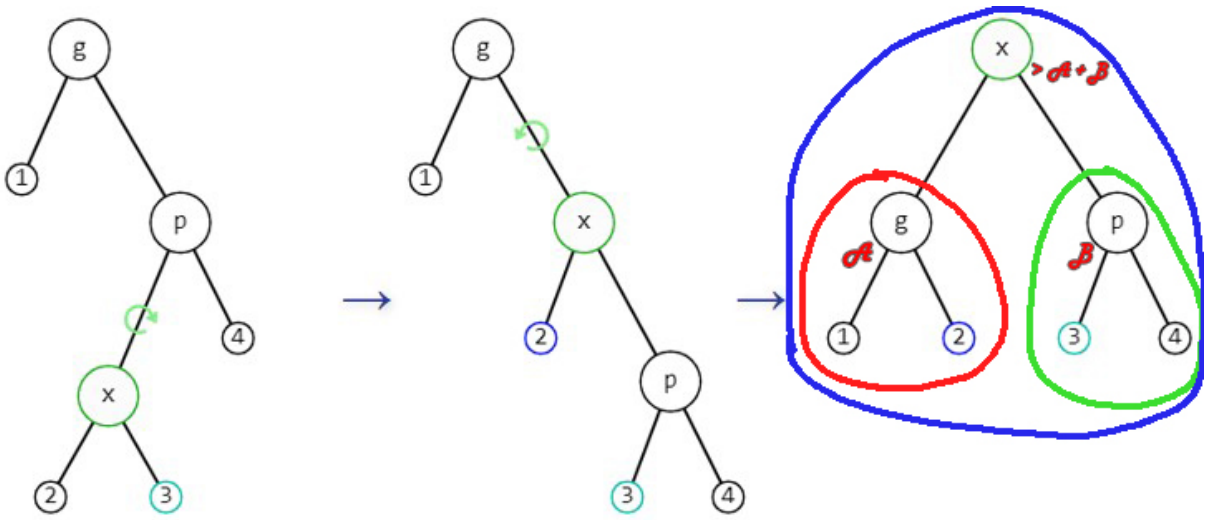
$$2r'(x) - 2r(x) - r'(p) - r'(g) + 2r(x) \leq -2$$

$$-2r'(x) + r'(p) + r'(g) \leq -2$$

Вспомним, что ранги это логарифм от размера дерева. $r'(p)$ – логарифм размера дерева p ; $r'(g)$ – логарифм размера дерева g ; $r'(x)$ – логарифм размера дерева x .

⁶Если $r(p)$ больше $r(x)$, то $-r(p)$ меньше $-r(x)$.

⁷Изменили коэффициент с 3 на 2 для следующих удобных преобразований. В таком случае неравенство стало ещё более строгим, нет ничего незаконного.



Вместо рангов подставляем логарифмы, получаем следующее:

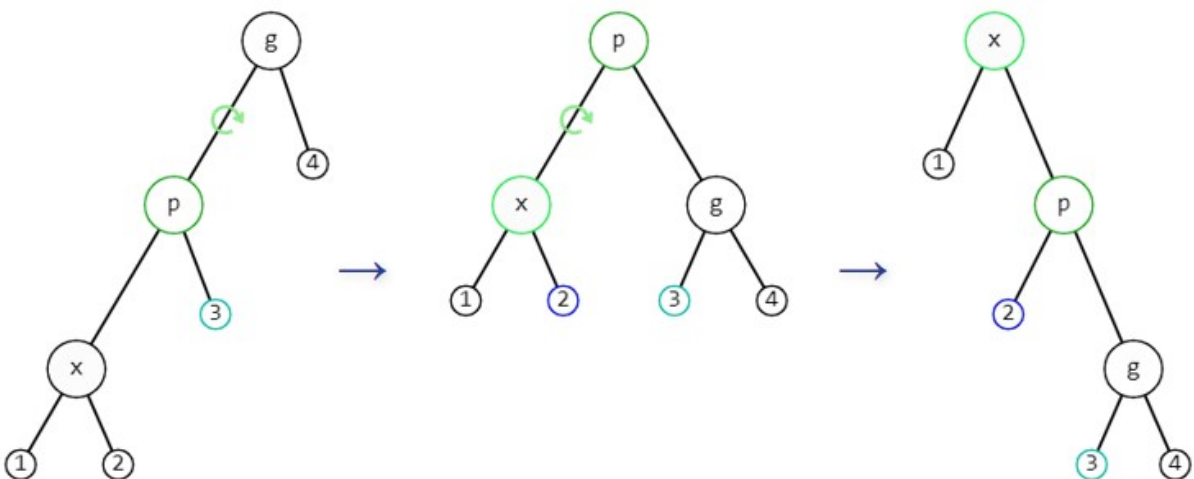
$$\log_2\left(\frac{\alpha}{\alpha + \beta} * \frac{\beta}{\alpha + \beta}\right) \leq -2$$

$$\log_2\left(\frac{\alpha}{\alpha + \beta} * \frac{\beta}{\alpha + \beta}\right) \leq \log_2 \frac{1}{4}$$

$$\frac{\alpha}{\alpha + \beta} * \frac{\beta}{\alpha + \beta} \leq \frac{1}{4}$$

Данное неравенство верно, так как произведение этих двух дробей не может быть больше $\frac{1}{4}$. Изначальное утверждение доказано.

3.3 Операция zig-zig

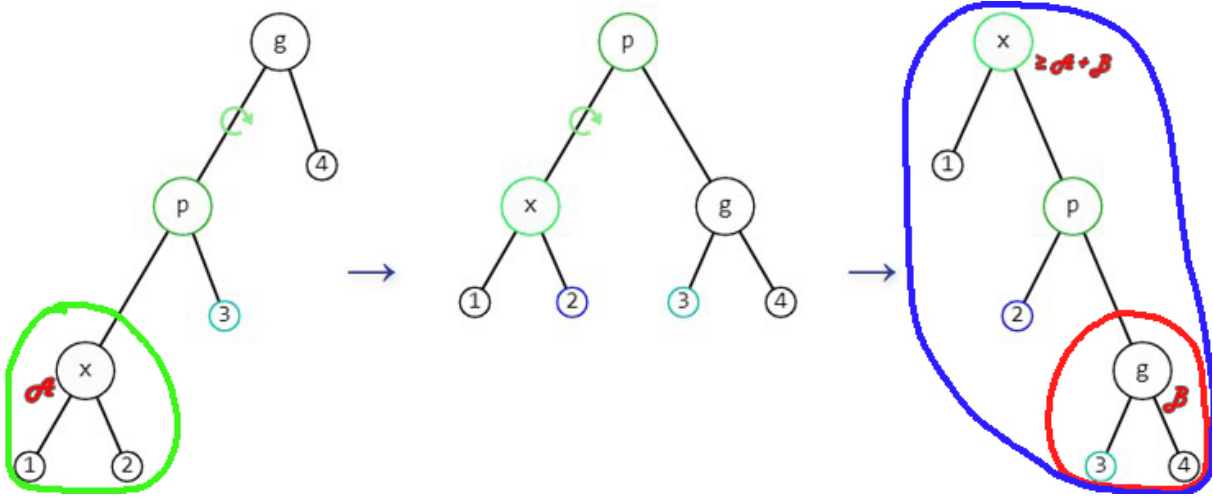


$\tilde{T} = 2 + (r'(x) + r'(p) + r'(g) - r(x) - r(p) - r(g))$. $r'(x)$ и $r(g)$ сокращаются; $-r(p)$ заменяется на $-r(x)$; $r'(p)$ заменяется на $r'(x)$.

Выходит: $\tilde{T} = 2 + (r'(x) + r'(g) - 2r(x))$. Докажем, что это выражение $\leq 3(r'(x) - r(x))$.

$$r(x) + r'(g) - 2r'(x) \leq -2$$

$r(x)$ – логарифм размера дерева x (до преобразований); $r'(g)$ – логарифм размера дерева g ; $r'(x)$ – логарифм размера дерева x .



$$\log_2\left(\frac{\alpha}{\alpha + \beta} * \frac{\beta}{\alpha + \beta}\right) \leq -2$$

$$\log_2\left(\frac{\alpha}{\alpha + \beta} * \frac{\beta}{\alpha + \beta}\right) \leq \log_2 \frac{1}{4}$$

$$\frac{\alpha}{\alpha + \beta} * \frac{\beta}{\alpha + \beta} \leq \frac{1}{4}$$

Данное неравенство верно, так как произведение этих двух дробей не может быть больше $\frac{1}{4}$. Изначальное утверждение доказано.

3.4 Общее заключение

Таким образом, мы разобрали три случая.

Изначально мы говорили, что $\tilde{T}(\text{splay}(x)) \leq 1 + 3(r'(x) - r(x))^8$. Теперь, когда были изучены и проанализированы все ситуации, мы с уверенностью можем сказать

$$\tilde{T}(\text{splay}) = O(\log n)$$

⁸ $r'(x) = \log_2 n$ и $r(x) = \log_2 S(x)$

Заключение

В ходе курсовой работы были выполнены поставленные цели: изучены принципы СД «Splay дерево»; реализованы операции для взаимодействия; проанализирована сложность алгоритмов структуры данных. Пришлось столкнуться с большим рядом проблем, начиная от повторения материалов математического анализа, заканчивая нехваткой знаний языка программирования C++, которые тоже пришлось пополнять благодаря различным источникам.

Разумеется, было приобретено много полезных знаний как в предметной области, так и за её пределами. Останавливаться на этом, конечно, не нужно. Настоятельно рекомендую самостоятельное изучение с помощью материалов из списка литературы (особенно англоязычных статей). Алгоритмы и структуры данных никогда не будут стоять на месте, так почему же не следовать за развивающимся будущим прямо сейчас?

Список использованной литературы

1. «АиСД S02E07. Splay дерево» 2021г.
Режим доступа: <https://youtu.be/cILoJnFhGV0>
2. «АиСД S02E07. Splay дерево» 2022г.
Режим доступа: <https://youtu.be/RmbLpFBqPqo>
3. «Splay Trees - Data Structures in 5 Minutes»
Режим доступа: <https://youtu.be/So8szqIvIFs>
4. «Splay дерево»
Режим доступа: <https://neerc.ifmo.ru/wiki/index.php?title=Splay-дерево>
5. «Splay tree»
Режим доступа: https://en.wikipedia.org/wiki/Splay_tree
6. «Splay деревья»
Режим доступа: <https://habr.com/ru/company/JetBrains-education/blog/210296/>
7. «Amortized analysis»
Режим доступа: https://en.wikipedia.org/wiki/Amortized_analysis
8. «Performance Analysis of BSTs in System Software»
Режим доступа: <https://benpfaff.org/papers/libavl.pdf>
9. «Binary search tree»
Режим доступа: https://en.wikipedia.org/wiki/Binary_search_tree

Приложение

```
1  template <typename T>
2  class SplayTree {
3      private:
4          // tree node structure
5          struct Node {
6              Node *leftChild; // pointer to left child
7              Node *rightChild; // pointer to right child
8              Node *parent; // pointer to parent
9              T data; // data field
10
11              // constructor
12              Node(const T &data)
13                  : leftChild(nullptr), rightChild(nullptr), parent(nullptr),
14                    data(data) {}
15
16              // destructor
17              ~Node() = default;
18
19              *root;
20
21              // updates pointers to child (for grandpa) and parent (for child)
22              // seems like swap, but after that necessary link old child and old parent
23              void updateParents(Node *parent, Node *child) {
24                  // update child's grandpa's pointer
25                  if (parent->parent == nullptr) {
26                      root = child;
27                  } else if (parent == parent->parent->leftChild) {
28                      parent->parent->leftChild = child;
29                  } else if (parent == parent->parent->rightChild) {
30                      parent->parent->rightChild = child;
31                  }
32
33                  // update child pointer
34                  if (child) {
35                      child->parent = parent->parent;
36                  }
37              }
38
39              // performs left rotation
40              // (rotation of the edge connecting n and it's right child)
41              void leftRotate(Node *n) {
42                  Node* rightChild = n->rightChild;
43
44                  // execute shift left child of the n's right child to n's right child
45                  // (even if equal nullptr)
46                  n->rightChild = rightChild->leftChild;
47                  if (rightChild->leftChild) {
48                      rightChild->leftChild->parent = n;
49                  }
50              }
51          };
52  }
```

```

48     updateParents(n, rightChild);
49
50     // link old child and old parent
51     rightChild->leftChild = n;
52     rightChild->leftChild->parent = rightChild;
53 }
54
55 // performs right rotation
56 // (rotation of the edge connecting n and it's left child)
57 void rightRotate(Node *n) {
58     Node *leftChild = n->leftChild;
59
60     // execute shift right child of the n's left child to n's left child
61     // (even if equal nullptr)
62     n->leftChild = leftChild->rightChild;
63     if (leftChild->rightChild) {
64         leftChild->rightChild->parent = n;
65     }
66
67     updateParents(n, leftChild);
68
69     // link old child and old parent
70     leftChild->rightChild = n;
71     leftChild->rightChild->parent = leftChild;
72 }
73
74 // performs splay
75 // (delivers a node n to the root of the tree using left and right pivots)
76 void splay(Node *n) {
77     while (parent(n)) {
78         // case: node n is left child
79         if (n == parent(n)->leftChild) {
80             if (!grandparent(n)) { // there isn't grandparent
81                 // zig operation
82                 rightRotate(parent(n));
83             } else if (grandparent(n)->leftChild == parent(n)) {
84                 // n's parent - left child
85                 // zig-zig operation
86                 rightRotate(grandparent(n));
87                 rightRotate(parent(n));
88             } else { // n's parent - right child
89                 // zig-zag operation
90                 rightRotate(parent(n));
91                 leftRotate(parent(n));
92             }
93         } else { // case: node n is right child
94             if (!grandparent(n)) { // there isn't grandparent
95                 // zig operation
96                 leftRotate(parent(n));
97             } else if (grandparent(n)->rightChild == parent(n)) {
98                 // n's parent - right child
99                 // zig-zig operation
100                 leftRotate(grandparent(n));
101                 leftRotate(parent(n));
102             } else { // n's parent - left child
103                 // zig-zag operation
104                 leftRotate(parent(n));
105                 rightRotate(parent(n));
106             }
107         }

```

```

108     }
109 }
110 }
111
112 // returns parent of the n node
113 Node* parent(const Node *n) const {
114     return n->parent;
115 }
116
117 // returns grandparent of the n node
118 Node* grandparent(const Node *n) const {
119     return n->parent->parent;
120 }
121
122 // returns minimum from subtree with root n
123 Node* _minimum(Node *n) const {
124     Node *minimum = n;
125     while (minimum->leftChild) {
126         minimum = minimum->leftChild;
127     }
128     return minimum;
129 }
130
131 // returns maximum from subtree with root n
132 Node* _maximum(Node *n) const {
133     Node *maximum = n;
134     while (maximum->rightChild) {
135         maximum = maximum->rightChild;
136     }
137     return maximum;
138 }
139 public:
140 SplayTree() : root(nullptr) {}
141
142 // inserts an element with data insertingData
143 void insert(const T &insertingData) {
144     Node *insertPlace = root;
145     Node *parentOfInsertedPlace = nullptr;
146
147     // finding suitable place
148     while (insertPlace) {
149         // necessary save previous insertPlace
150         // (it will be a parent of the insertPlace)
151         parentOfInsertedPlace = insertPlace;
152         if (insertingData > insertPlace->data) {
153             insertPlace = insertPlace->rightChild;
154         } else {
155             insertPlace = insertPlace->leftChild;
156         }
157     }
158
159     // creating inserting element
160     Node *insertedNode = new Node(insertingData);
161     insertedNode->parent = parentOfInsertedPlace;
162
163     // link inserted element's parent with him
164     if (!parentOfInsertedPlace) {
165         root = insertedNode;
166     } else if (parentOfInsertedPlace->data < insertedNode->data)
{

```

```

167         parentOfInsertedPlace->rightChild = insertedNode;
168     } else {
169         parentOfInsertedPlace->leftChild = insertedNode;
170     }
171
172     splay(insertedNode);
173 }
174
175 // returns a pointer to the node with data searchingData
176 Node* find(const T &searchingData) {
177     Node* findingNode = root;
178
179     while (findingNode) {
180         if (searchingData > findingNode->data) {
181             findingNode = findingNode->rightChild;
182         } else if (searchingData < findingNode->data) {
183             findingNode = findingNode->leftChild;
184         } else {
185             splay(findingNode);
186             return findingNode;
187         }
188     }
189
190     return nullptr;
191 }
192
193 // removes an element with data deletingData
194 void erase(const T &deletingData) {
195     Node* deletingNode = find(deletingData);
196
197     if (deletingNode) {
198         if (deletingNode->rightChild == nullptr) {
199             updateParents(deletingNode, deletingNode->leftChild);
200         } else if (deletingNode->leftChild == nullptr) {
201             updateParents(deletingNode, deletingNode->rightChild);
202         } else { // case: there are two children owned deletingNode
203             Node* replacementRemotedNode = _maximum(deletingNode
204             ->leftChild);
205
206             // case: replacing node can have subtrees
207             if (parent(replacementRemotedNode) != deletingNode) {
208                 updateParents(replacementRemotedNode,
209                 replacementRemotedNode->leftChild);
210
211                 replacementRemotedNode->leftChild = deletingNode
212                 ->leftChild;
213                 replacementRemotedNode->leftChild->parent =
214                 replacementRemotedNode;
215             }
216
217             updateParents(deletingNode, replacementRemotedNode);
218
219             replacementRemotedNode->rightChild = deletingNode->
220             rightChild;
221             deletingNode->rightChild->parent =
222             replacementRemotedNode;
223         }
224
225         if (parent(deletingNode)) {

```

```

220         splay(parent(deletingNode));
221     }
222     delete deletingNode;
223 }
224 }
225
226 // merges this tree with tree addedTree
227 void merge(const SplayTree &addedTree) {
228     // necessary make sure that all keys of the current (this) tree less
229     // than the keys of the tree T2
230     // and remember "smallest" tree
231     SplayTree smallest, largest;
232     if (this->root->data > addedTree.root->data) {
233         smallest.root = addedTree.root;
234         largest.root = this->root;
235     } else {
236         smallest.root = this->root;
237         largest.root = addedTree.root;
238     }
239
240     smallest.splay(_maximum(smallest.root));
241
242     smallest.root->rightChild = largest.root;
243     this->root = smallest.root;
244 }
245
246 // splits the tree into two trees
247 // first tree going to be in "this" tree, second in secondTree
248 // first tree contains elements less or equal than splitElement
249 // second tree contains elements greater than x
250 // note: not necessary splitElement there's in the tree
251 void split(SplayTree &secondTree, const T &splitElement) {
252     Node* similarOrEqualToSplitElement = root;
253
254     // finding element, which similar to splitElement (even can be equal)
255     while (similarOrEqualToSplitElement) {
256         if (splitElement < similarOrEqualToSplitElement->data &&
257             similarOrEqualToSplitElement->leftChild) {
258             similarOrEqualToSplitElement =
259             similarOrEqualToSplitElement->leftChild;
260         } else if (splitElement > similarOrEqualToSplitElement->
261             data && similarOrEqualToSplitElement->rightChild) {
262             similarOrEqualToSplitElement =
263             similarOrEqualToSplitElement->rightChild;
264         } else {
265             break;
266         }
267     }
268
269     splay(similarOrEqualToSplitElement);
270
271     secondTree.root = similarOrEqualToSplitElement->rightChild;
272     similarOrEqualToSplitElement->rightChild = nullptr;
273 }
274
275 // returns minimum of the tree
276 T minimum() const {
277     return _minimum(root->data;

```

```
276 // returns maximum of the tree
277 T maximum() const {
278     return _maximum(root)->data;
279 }
280
281 // returns "true if tree is empty, otherwise returns "false"
282 bool empty() const {
283     return (root == nullptr);
284 }
285 };
```